

## **Ejercicios evaluables UA6**

Esta actividad aborda los criterios de evaluación del RA.

### **Instrucciones**

- Todas las actividades que se realicen deben entregarse mediante un enlace al repositorio de Github y un documento (si se pide alguna evidencia o se plantean preguntas): DAW\_DWES-UA6-ApellidosNombre.pdf
- Las entregas deberán realizarse en la tarea correspondiente en el formato especificado antes de la hora de finalización. Cualquier tarea que no cumpla estos requisitos tendrá una calificación de 0.
- Se pedirá que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de la tarea será de 0.

### **Sobre el uso de IA y fuentes externas**

Puedes utilizar herramientas como buscadores o inteligencia artificial como apoyo, pero no como sustituto de tu trabajo. Las respuestas deben estar redactadas por ti, con tus propias ideas y razonamientos.

Se valorará especialmente el análisis personal y la justificación razonada.

### **Ejercicio 1**

Un usuario quiere añadir un videojuego a su biblioteca, indicando el identificador del juego (del catálogo externo), el estado inicial y las horas jugadas.

#### Dónde se hace

- Ruta: POST /api/library/entries/

#### Qué envía el frontend

La petición debe traer JSON obligatorio con estos campos:

- *external\_game\_id* (string)
- *status* (string) con uno de estos valores: wishlist, playing, completed, dropped
- *hours\_played* (integer) y debe ser  $\geq 0$

#### Comprobaciones que debes hacer antes de aceptar

Tu backend debe revisar los datos que recibe:

- JSON válido: Si llega un JSON vacío {} → error
- Están todos los campos con los tipos correctos: external\_game\_id, status, hours\_played
- status está dentro de los valores permitidos y hours\_played es  $\geq 0$

#### Respuesta si todo va bien

Si la petición es correcta, se crea la entrada y se devuelve:

- Código: 201
- Body: JSON con los campos del recurso creado: id, external\_game\_id, status, hours\_played

#### Respuesta si algo falla

Si hay cualquier problema de validación:

- Código: 400
- Body: el error debe seguir el contrato definido en el ejercicio 2 (puedes dejar esto pendiente hasta que abordes el ejercicio 2).

#### Qué debes probar

- 1 ejemplo correcto (que devuelva 201)
- Al menos 5 ejemplos incorrectos (que devuelvan 400), bien escogidos para cubrir errores distintos: status no sea string, status no esté en alguno de los valores especificados, hours\_played no sea integer, hours\_played sea menor que 0..., body vacío, etc

## Ejercicio 2

Cuando el frontend envía datos incorrectos, necesita que el backend responda siempre con el mismo formato, para poder mostrar y gestionar esos errores de forma estable.

### Dónde se aplica

Este ejercicio no es una ruta concreta: se aplica a todas las rutas de esta semana (y de las próximas) cuando haya un error de validación.

### Qué debe devolver el backend en el error 400

Debe devolver exactamente este JSON:

```
{
  "error": "validation_error",
  "message": "Datos de entrada inválidos",
  "details": { "campo": "motivo" }
}
```

Tiene los siguientes elementos:

- ☐ error: Es una cadena que identifica el error. A lo largo de los ejercicios definiremos cuáles son esos valores. De momento, `validation_error`.
- ☐ message: Una cadena (en español) que da más información sobre el error.
- ☐ details: Un diccionario donde la clave es el campo y el valor el motivo. Recuerda que pueden existir varios campos que den un error. Es opcional.

Nota: A partir de ahora habrá otros códigos de error (`duplicate_entry`, `unauthorized`, etc.). El contrato de 400 `validation_error` es para errores de validación de campos/JSON; `duplicate_entry` se devolverá cuando en una vista se intente crear un campo que esté duplicado.

### Cuando se debe responder con este formato

Cuando se quiera devolver un código de estado 400 en cualquiera de estos casos:

- ☐ JSON mal formado
- ☐ JSON vacío (`{}`)
- ☐ faltan campos obligatorios
- ☐ tipos incorrectos
- ☐ Los campos no tienen un dominio permitido (`hours_played` menor que 0, `status` no tiene un valor permitido...)

### Restricción importante

- ☐ No se devuelve HTML ni texto plano, siempre JSON
- ☐ No se usan formatos distintos según el fallo (siempre el mismo contrato)

**Ejercicio 3**

Un usuario intenta añadir dos veces el mismo juego (mismo *external\_game\_id*) a su biblioteca. El backend debe detectarlo e impedirlo devolviendo un código 400.

Dónde se hace

- ☐ Ruta: POST /api/library/entries/

Regla principal

- ☐ No puede existir más de una entrada con el mismo *external\_game\_id*

Respuesta si alguien intenta duplicar

Si se intenta crear un juego en la biblioteca con un *external\_game\_id* que ya existe:

- ☐ Código: 400
- ☐ Body (con este formato):

```
{
  "error": "duplicate_entry",
  "message": "El juego ya existe en la biblioteca",
  "details": { "external_game_id": "duplicate" }
}
```

Qué no se debe hacer

- ☐ No se sobrescribe el juego en la base de datos
- ☐ No debe devolver un error 500 por excepciones no controladas

Qué debes probar

- ☐ 1 creación correcta → 201
- ☐ 1 intento de duplicado → 400 con el JSON anterior

**Ejercicio 4**

El frontend necesita dos cosas: un listado y un detalle, y ambos deben usar el mismo formato de datos. Además, si pide un id que no existe, el backend debe responder de forma clara.

Dónde se hace

- ☐ Ruta listado: GET /api/library/entries/
- ☐ Ruta detalle: GET /api/library/entries/{id}/

Respuesta del listado

- ☐ Código: 200

- Body: una lista de elementos, donde cada entrada tiene exactamente estos campos: *id*, *external\_game\_id*, *status*, *hours\_played*

#### Respuesta del detalle

- Código: 200
- Body: un objeto con exactamente estos campos: *id*, *external\_game\_id*, *status*, *hours\_played*

Si el {id} no existe

- Código: 404
- Body:

```
{
  "error": "not_found",
  "message": "La entrada solicitada no existe"
}
```

#### Restricción importante

- No devolver campos distintos en listado y detalle
- No devolver HTML en errores

#### Qué debes probar

- Crear al menos 2 entradas en la biblioteca
- Probar:
  - listado (200)
  - detalle de una existente (200)
  - detalle de una inexistente (404 con el JSON exacto)

### **Ejercicio 5**

Un usuario quiere actualizar el juego conforme progresa: cambiar el estado y/o las horas jugadas. Esta operación debe reutilizar las mismas validaciones y el mismo estilo de errores.

#### Dónde se hace

- Ruta: PATCH /api/library/entries/{id}/

#### Qué envía el frontend

La petición debe incluir un JSON obligatorio y no vacío con: status y/o hours\_played

#### Comprobaciones que debes hacer antes de aceptar

Tu vista debe revisar:

- JSON válido

- ❑ Body no vacío ({} → error)
- ❑ Solo se aceptan status y hours\_played (si llega cualquier otro campo → error)
- ❑ Tipos correctos y dominios de los campos (status dentro de los valores permitidos y hours\_played  $\geq 0$ )

#### Respuestas posibles

- ❑ Si {id} no existe:
  - Código: 404
  - Body: error con código not\_found
- ❑ Si hay error de validación:
  - Código: 400
  - Body: error con código validation\_error
- ❑ Si todo va bien:
  - Código: 200
  - Body: JSON con información del recurso actualizado con: *id*, *external\_game\_id*, *status*, *hours\_played*

#### Qué debes probar

- ❑ 1 caso correcto (200)
- ❑ Varios casos incorrectos (400) que cubran errores distintos (por ejemplo body vacío, status inválido, hours\_played negativo, campo desconocido...)
- ❑ 1 caso con {id} inexistente (404)

**Rúbrica de evaluación de la tarea**

| Criterio de evaluación | Nivel alto<br>(9-10) | Nivel medio<br>(5-9) | Nivel Bajo<br>(1-5) | No logrado<br>(0) |
|------------------------|----------------------|----------------------|---------------------|-------------------|
|                        |                      |                      |                     |                   |
|                        |                      |                      |                     |                   |
|                        |                      |                      |                     |                   |
|                        |                      |                      |                     |                   |